

Is AI actually working?

Here's how to find out, especially if you're a developer

Jayesh Shinde | editor@digit.in

Every other week, newer AI models keep topping the benchmarking leaderboard, busy one-upping one another. But high scores don't tell you if a product built around an AI model will work for a real person or business. Spurthi Tallam, a senior software engineer (Applied AI/ML) at New York-based resale marketplace LePrix, studies when an AI system should act on its own and when it should hand the decision to a person. According to her, the real test of an AI model boils down to whether it improves outcomes where people actually use it. Also if its mistakes are acceptable, visible and fixable. Edited excerpts of my interview follows:

Q AI models keep getting higher benchmark scores. For developers, what does a benchmark tell us, and what does it not?

A Spurthi: A benchmark is a standardized test. It presents the same set of questions or tasks to different models, allowing us to compare their performance under controlled conditions. A benchmark score is not a product guarantee. It does not predict how the model will perform with your documents, customers, terminology, or edge cases. Stanford's HELM research highlighted this limitation: meaningful evaluation requires multiple scenarios and measures, not one headline number.



Public test material can also make its way into training data, or developers may overoptimize for a well-known test. Peer-reviewed research on benchmark contamination suggests a high score may sometimes reflect familiarity with the exam rather than consistent performance in a new environment.

So I would treat a benchmark as evidence of capability, not proof of usefulness. The first question I'd ask is: what was tested, and how closely does it match what I need this product to do?

Q Models claim to be better at reasoning, coding and using tools. How do you tell whether that matters for your everyday tasks?

A Spurthi: Start with your work, not the model announcement. Take a small set of tasks that represent what you do: a routine case, a difficult case, an ambiguous case, and a few examples where a mistake would matter. Then compare the new system with whatever you use today, whether an earlier model, standard software or a manual process.

The phrase "better than what?" is essential here. A model may generate a stronger answer than its predecessor but still fail to improve the overall workflow. If an employee saves five minutes drafting something but spends ten minutes correcting it, the model has improved while the job has not.

For coding, I would look beyond simple code generation. Can the system understand an existing codebase, make the necessary changes, and pass the tests without breaking anything else? SWE-bench, a research project from Princeton and the University of Chicago,